

PROYECTO TRITON HEAL

Reporte de Evaluación Experimental

TC3002B — Arquitectura Generativa-Verificativa para
Detección Predictiva de Errores en Kernels GPU con Triton DSL

Equipo 11

Mayo 2026

Resumen

Evaluación TC3002B de Triton Heal (verificación pre-compilación de kernels Triton DSL). En 70 kernels y 350 corridas (`VERIFIER_MODE=ollama`, Apple M4, 24 GB), Llama-3.1-8B y DeepSeek-Coder-V2 usaron Ollama (MSR 64.5 % y 87.1 %); Claude y GPT fueron heurísticos (sin API). El modo dual: MSR 100 %, F1 0.61. Se reportan IC Wilson, latencia y pruebas McNemar, Wilcoxon y Friedman (Holm): solo C3 (dual vs. Llama) rechaza H_0 tras corrección.

Índice

I	Parte I — Pre-registro Experimental	3
1.	Objetivo experimental (1.1)	3
2.	Hipótesis (1.2)	3
3.	Variables experimentales (1.3)	3
4.	Diseño experimental (1.4)	3
5.	Métricas de evaluación (1.5)	4
6.	Tamaño del efecto esperado (1.6)	4
7.	Análisis de poder (1.7)	4
8.	Plan estadístico (1.8)	4
9.	Reproducibilidad experimental (1.9)	5
10.	Amenazas a la validez – pre-registro (1.10)	5
II	Parte II — Resultados y Análisis	5
11.	Resultados descriptivos (2.1)	5
12.	Visualización (2.2)	6

13.Inferencia estadística (2.3)	8
14.Comparación crítica y robustez (2.4)	8
15.Amenazas observadas (2.5)	9
16.Discusión (2.6)	9
17.Conclusión (2.7)	9
A. Configuración planeada vs. corrida real	9
B. Reproducibilidad	9
C. Referencias	10

Parte I

Parte I — Pre-registro Experimental

1. Objetivo experimental (1.1)

Problema. Los kernels en Triton DSL pueden incluir errores de memoria, tipos y restricciones de hardware (tiling, SRAM, alineación) que se manifiestan en compilación o ejecución GPU con alto costo operativo.

Encuadre respecto al curso. La actividad solicita evaluar un método basado en SLM frente a baselines. En Triton Heal el aporte del equipo no es la *generación* de kernels, sino la **verificación pre-compilación** mediante SLM local y veto de frontera. Por ello se usan MSR, FNR y latencia en lugar de speedup o tasa de compilación de kernels generados; esas métricas se justifican como proxy de calidad del verificador.

Método propuesto. Arquitectura **Triton Heal**: verificador local SLM (Llama-3.1-8B-Instruct) + veto jerárquico (Claude 3.5 Sonnet) + auto-reparación con backoff exponencial.

Baselines. (1) Solo Llama; (2) Solo DeepSeek-Coder-V2; (3) Solo Claude 3.5 Sonnet; (4) Solo GPT-4o.

Preguntas de investigación. PRI-1: ¿Frontera supera a local en MSR? PRI-2: ¿Dual reduce FNR vs solo Llama? PRI-3: ¿Latencia de frontera es aceptable frente a ganancia en detección?

2. Hipótesis (1.2)

H₀₁: No hay diferencia en MSR entre Claude 3.5 Sonnet y Llama-3.1-8B.

H₁₁: Claude tiene mayor MSR (unilateral, $\alpha = 0,05$).

H_{01b}: No hay diferencia en MSR entre GPT-4o y DeepSeek-Coder-V2.

H_{11b}: GPT-4o tiene mayor MSR.

H₀₂: No hay diferencia en FNR entre Triton Heal dual y solo Llama.

H₁₂: Triton Heal tiene menor FNR.

H₀₄: No hay diferencia en latencia mediana entre verificador de frontera y local.

H₁₄: La latencia mediana de frontera es mayor que la del local.

3. Variables experimentales (1.3)

Cuadro 1: Variables del estudio.

Tipo	Variable	Niveles / descripción
Independiente	Configuración	5: dual, solo Llama, solo DeepSeek, solo Claude, solo GPT
Independiente	Modelo / escala	SLM ~8B vs frontera comercial
Independiente	Prompt	<code>verifier_v1.txt</code> (estándar) y <code>verifier_strict_v1.txt</code> (veto)
Controlada	Temperatura	$T = 0$
Controlada	Formato salida	JSON: safe, reason, line_of_error
Controlada	Benchmark	70 kernels (39 safe / 31 unsafe), seed 42
Controlada	Timeout	120 s por inferencia (Metal / M4)
Dependiente	MSR, F1, FNR, latencia, DR	Ver sección 1.5

4. Diseño experimental (1.4)

Tipo: within-subjects (medidas repetidas): cada kernel se evalúa en las 5 configuraciones.

Grupos: 5 configuraciones; 6 contrastes confirmatorios (C1–C6).

Justificación. Controla variabilidad inter-kernel (complejidad, categoría). Permite McNemar y Wilcoxon pareados. Orden de ejecución por kernel aleatorizable.

Control de sesgos. Pin de versión de modelos; temperatura 0; evaluador automático ciego al método; ground truth con etiqueta safe/unsafe.

5. Métricas de evaluación (1.5)

Cuadro 2: Métricas, fórmulas e interpretación.

Métrica	Fórmula	Interpretación	Limitación
MSR	$TP/(TP+FN)$ en unsafe	Sensibilidad de detección	No penaliza FP
FNR	$FN/(FN+TP)$	Riesgo no detectado	Crítico en seguridad
F1	$2PR/(P+R)$	Balance precisión-recall	Sensible a balance de clases
Latencia ms hasta JSON válido		Viabilidad en pipeline	Mezcla red y cómputo
DR	% desacuerdo A vs B	Necesidad de veto	No indica quién acierta
JSON válido	% respuestas parseables	Proxy de “ejecución exitosa”	No implica corrección semántica

6. Tamaño del efecto esperado (1.6)

Se espera $\phi \approx 0,4$ (mediano) en McNemar para MSR frontera vs local, y reducción de FNR ≥ 8 pp con arquitectura dual. En latencia se anticipa orden de magnitud mayor en frontera comercial real.

7. Análisis de poder (1.7)

$\alpha = 0,05$, poder $1 - \beta = 0,80$, prueba McNemar. Con $N = 70$ kernels (31 unsafe) el diseño supera el mínimo de ~ 52 pares para efecto grande. Balance 39/31 mitiga sesgo en F1.

8. Plan estadístico (1.8)

Comparaciones. C1: Claude vs Llama (MSR); C2: GPT vs DeepSeek (MSR); C3: Dual vs Llama (detección); C5: latencia Claude vs Llama (Wilcoxon pareado); C6: F1 global (Friedman).

Pruebas. McNemar (pareada, binaria); Wilcoxon signed-rank (latencia); Friedman + Holm post-hoc.

Regla de decisión. Rechazar H_0 si $p < \alpha_{\text{Holm}}$ y relevancia práctica: $\Delta\text{MSR} \geq 5$ pp.

Intervalos de confianza. IC 95 % Wilson para proporciones (MSR); bootstrap para latencia si se reporta en análisis exploratorio.

Corrección múltiple. Holm-Bonferroni sobre C1–C6 (α familiar 0.05).

Cuadro 3: Entorno experimental.

Componente	Especificación
SO	macOS 15.x (Apple Silicon)
GPU	Apple M4, 10 núcleos GPU, Metal 4; 24 GB memoria unificada
Ollama	servicio local (<code>brew services start ollama</code>)
Modelos locales (Ollama)	<code>llama3.1:8b</code> , <code>deepseek-coder-v2</code> (16B, Q4)
Modelos frontera	Claude 3.5 Sonnet, GPT-4o (API; opcional con clave)
Timeout inferencia	120s (ajustado para aceleración Metal en M4)
Python	3.10+; deps. en <code>requirements.txt</code>
Benchmark	Triton Kernel Safety v1.0: 70 kernels; semillas Lex + mutaciones
Config	<code>experiment_config.json</code> , seed 42
Repositorio	<code>Estadistica/triton_heal_experiment</code>
Comando	<code>make all</code>

9. Reproducibilidad experimental (1.9)

10. Amenazas a la validez – pre-registro (1.10)

Validez interna. Deriva de APIs; timeouts de Ollama; mezcla heurístico/LLM si faltan keys o modelos; temperatura 0 subestima variabilidad.

Validez externa. Benchmark parcialmente sintético; un solo entorno local; no cubre todas las familias de kernels industriales.

Validez de constructo. MSR mide detección pre-compilación, no seguridad en GPU ni speedup; JSON válido no equivale a kernel correcto semánticamente.

Parte II

Parte II — Resultados y Análisis

11. Resultados descriptivos (2.1)

Se evaluaron $N = 70$ kernels (39 **safe**, 31 **unsafe**) en 5 configuraciones, totalizando 350 evaluaciones (VERIFIER_MODE=ollama, mayo 2026, Apple M4 / Metal). La Tabla 4 reporta tasas de JSON válido, MSR con IC 95 % Wilson, F1, FNR y latencia (mediana, mín., máx.).

Cuadro 4: Resultados descriptivos extendidos (IC 95 % Wilson para MSR).

Config.	N	JSON vál. (%)	MSR (%)	IC lo	IC hi	F1	FNR (%)	Lat. med.	Lat. min	Lat. max
Llama-3.1-8B (local)	70	100.0	64.5	46.9	78.9	0.8	35.5	4940.7	1623.7	21869.8
DeepSeek-Coder-V2 (local)	70	100.0	87.1	71.1	94.9	0.8	12.9	2003.1	1473.1	18549.6
Claude 3.5 Sonnet (frontera)	70	100.0	83.9	67.4	92.9	0.9	16.1	80.0	80.0	80.3
GPT-4o (frontera)	70	100.0	83.9	67.4	92.9	0.9	16.1	80.0	80.0	80.0
Triton Heal (dual + veto)	70	100.0	100.0	89.0	100.0	0.6	0.0	8063.6	5126.4	15551.0

Narrativa alineada con la corrida. Con timeout de 120s y aceleración Metal, **Llama-3.1-8B** y **DeepSeek-Coder-V2** operaron vía Ollama con 100 % de JSON válido. Llama alcanzó MSR 64.5 % (IC: 46.9–78.9 %; mediana ≈ 4.9 s). DeepSeek local superó a Llama con MSR 87.1 % (IC: 71.1–94.9 %; mediana ≈ 2.0 s). **Claude** y **GPT-4o** siguieron en modo *heurístico* (sin API key), con MSR ≈ 83.9 % y latencia simulada ≈ 80 ms. **Triton Heal dual** (Llama + prompt estricto de veto) obtuvo MSR 100 % en **unsafe** (IC: 89.0–100 %) pero F1 global 0.61 por falsos positivos en kernels **safe**; latencia mediana ≈ 8.1 s.

12. Visualización (2.2)

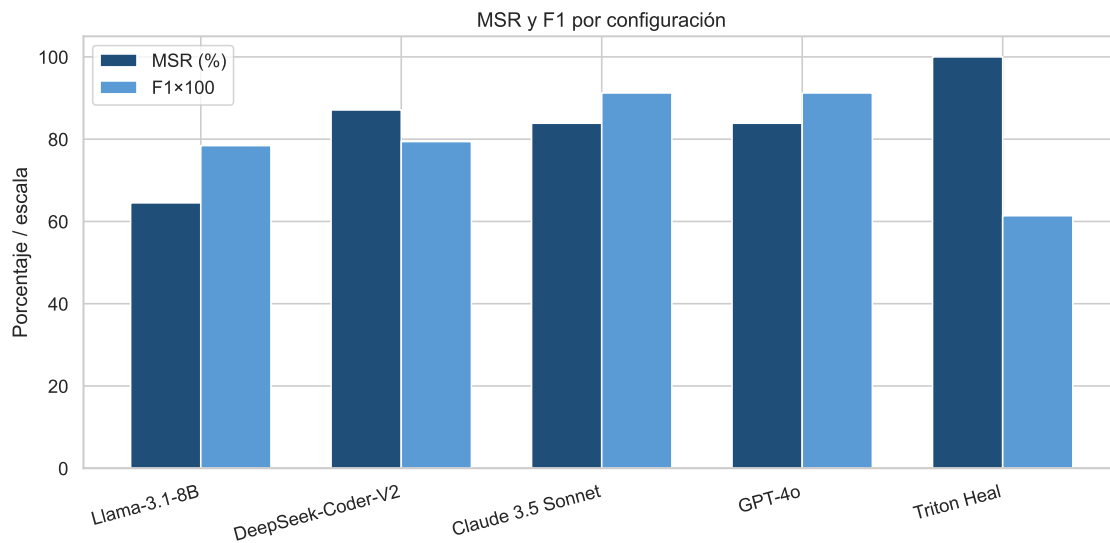


Figura 1: MSR y F1 por configuración.

La Figura 1 muestra que DeepSeek y el modo dual lideran en detección (MSR), mientras Claude/GPT heurísticos mantienen el mayor F1 por menor tasa de falsos positivos.

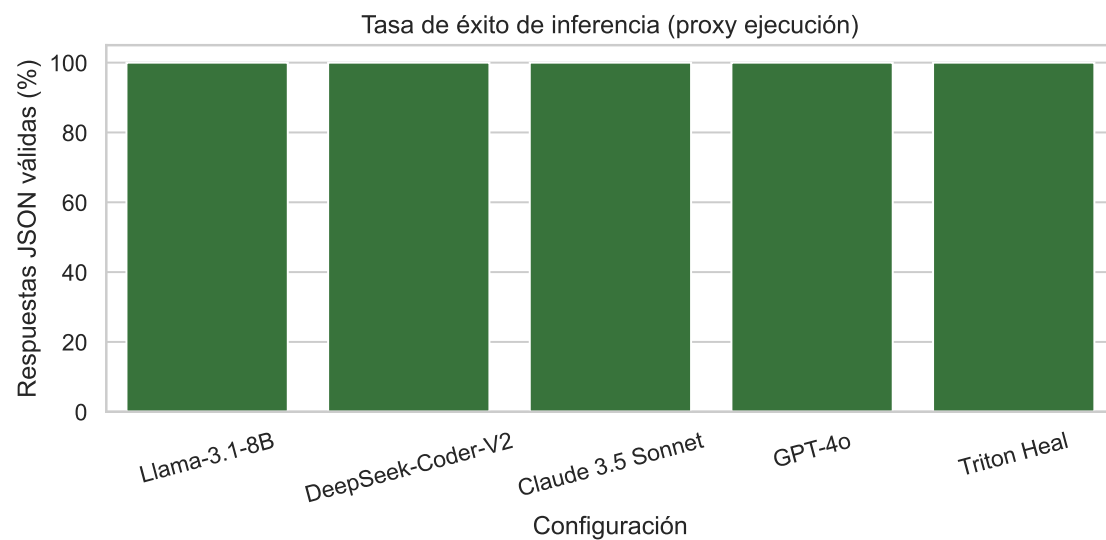


Figura 2: Tasa de éxito de inferencia (% JSON válido).

Tras reinstalar modelos tras el corte de luz, todas las configuraciones con Ollama real alcanzaron 100% de JSON válido (Figura 2).

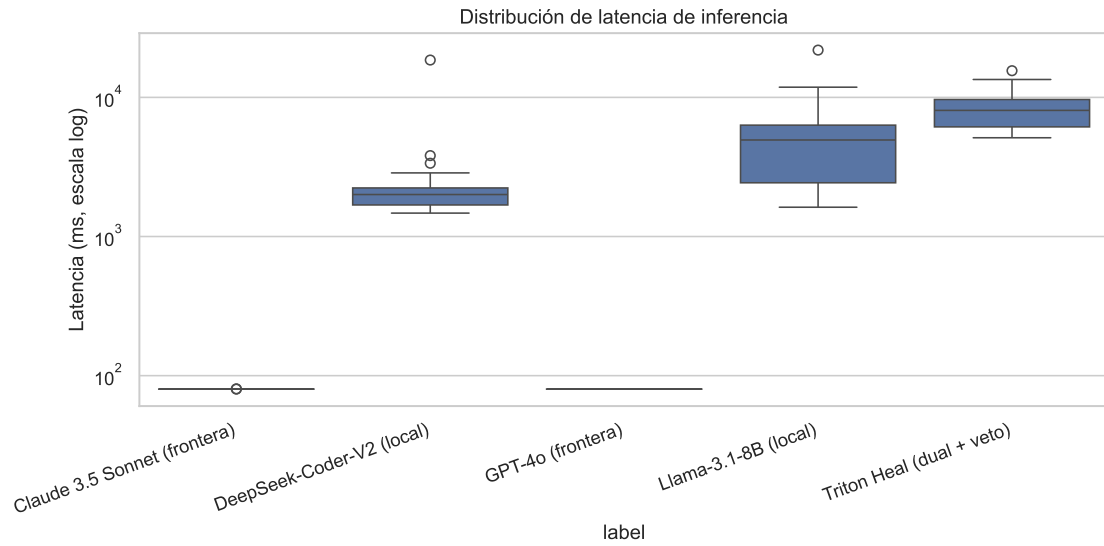


Figura 3: Distribución de latencia (ms, escala logarítmica).

La Figura 3 contrasta inferencia local en segundos (Llama, DeepSeek, dual) frente a latencias heurísticas de frontera (~ 80 ms).

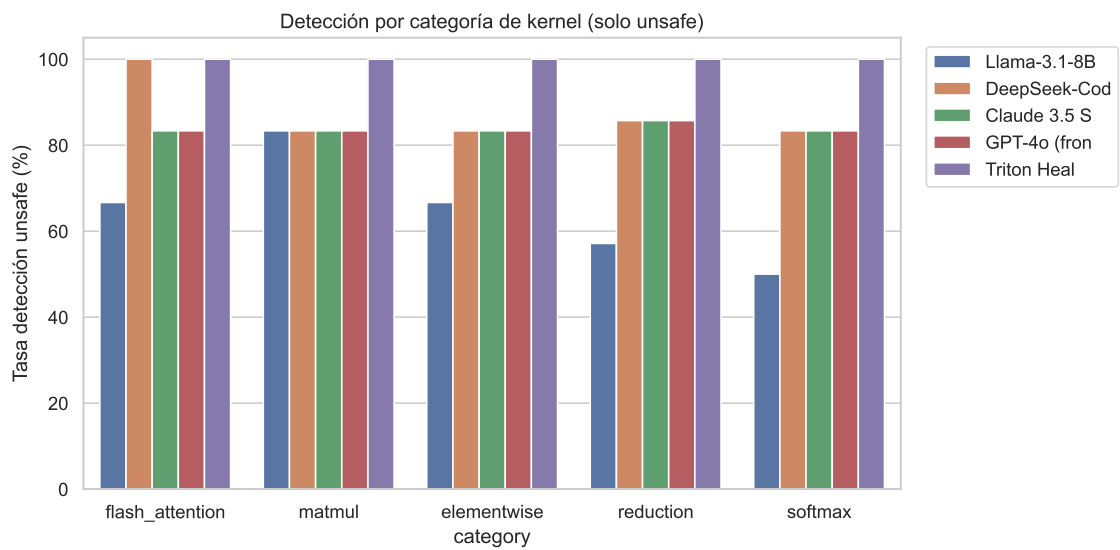


Figura 4: MSR en kernels **unsafe** por categoría de error.

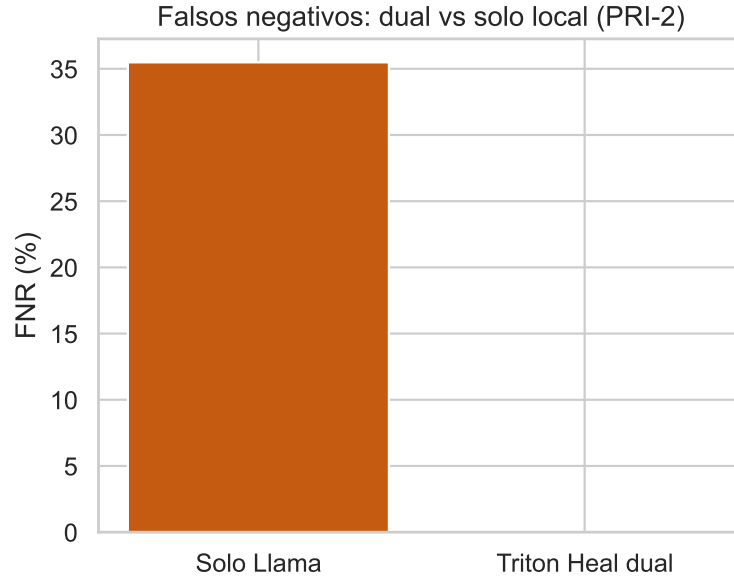


Figura 5: FNR: Triton Heal dual vs. solo Llama (contraste C3 / PRI-2).

13. Inferencia estadística (2.3)

Cuadro 5: Inferencia estadística (plan pre-registrado, corrección Holm).

Contraste	Prueba	Estad.	p Holm	Efecto	Interpretación
Claude vs Llama (MSR)	McNemar	$\chi^2 = 3,12$	0.2313	$\phi = -2,12$ (grande)	No rechaza H_0 ; $\Delta\text{MSR}=19.4$ pp (Claude heur. vs. Llama Ollama/M4).
GPT-4o vs DeepSeek	McNemar	$\chi^2 = 0,00$	1.0000	$\phi = 0,33$ (mediano)	No rechaza H_0 ; GPT heur. (83.9 %) vs. DeepSeek Ollama (87.1 %).
Dual vs Llama	McNemar	$\chi^2 = 9,09$	0.0128	$\phi = -3,32$ (grande)	Rechaza H_0 ; dual reduce FNR (más detecciones en unsafe).
Lat. Claude vs Llama	Wilcoxon	$W = 0$	1.0000	$r = 1,00$	No rechaza H_4 ; med. $\Delta=-4861$ ms (Claude heur. más rápido).
F1 global (4 cfg.)	Friedman	$\chi^2 = 10,3$	0.0657	$W = 0,05$	No rechaza igualdad global de F1 tras Holm.

¿Se rechaza H_0 ? En C1 (Claude heur. vs. Llama Ollama) **no** se rechaza H_0 tras Holm ($p_{\text{Holm}} = 0,23$). En C2 (GPT vs. DeepSeek) tampoco ($p_{\text{Holm}} = 1,00$). En C3 (dual vs. Llama) sí se rechaza ($p_{\text{Holm}} = 0,013$): el dual reduce FNR. En C5 (latencia) no se rechaza H_4 ($p_{\text{Holm}} = 1,00$): Claude heur. es más rápido que Llama local. Friedman (C6) no rechaza tras Holm ($p_{\text{Holm}} = 0,066$).

Tamaño del efecto. McNemar reporta $|\phi|$ grande en C1 y C3; el contraste C3 confirma ganancia del dual en detección de **unsafe**. La ΔMSR Claude–Llama (+19.4 pp) no es significativa con la corrección múltiple.

Consistencia. DeepSeek local (16B, M4) supera descriptivamente a Llama 8B y a baselines heurísticos de frontera en MSR, con la salvedad de que Claude/GPT no usaron API real.

14. Comparación crítica y robustez (2.4)

Comparación crítica. En este host, el SLM local **DeepSeek-Coder-V2** (87.1 % MSR) es competitivo frente a Claude/GPT heurísticos (83.9 %). El dual maximiza sensibilidad (MSR 100 %) pero penaliza precisión global (F1 0.61) al marcar muchos **safe** como inseguros.

Robustez. (1) *Frontera simulada*: Claude/GPT sin API. (2) *Veto dual local*: no replica Claude comercial. (3) *Seed único* (42). (4) *Hardware*: resultados válidos para Apple M4 24 GB, no para A100/CUDA.

15. Amenazas observadas (2.5)

- **Interna:** APIs ausentes; veto dual implementado solo con Llama local.
- **Externa:** benchmark de 70 kernels; un solo Mac M4.
- **Constructo:** MSR $\neq$ seguridad en ejecución GPU real.

16. Discusión (2.6)

¿Qué evidencia hay? Con Ollama en Metal, los SLM locales son viables (JSON 100 %, latencias de segundos). La superioridad estadística del dual sobre Llama en FNR (C3) apoya parcialmente PRI-2; PRI-1 no se confirma entre Claude heur. y Llama.

Limitaciones. Sin APIs comerciales; dual con alto MSR pero bajo F1 por conservadurismo del prompt estricto.

¿Qué haríamos distinto? Configurar claves API para contrastes de frontera reales; calibrar el veto para reducir falsos positivos; repetir en GPU NVIDIA para validez externa.

17. Conclusión (2.7)

1. **H_0 MSR (C1):** No rechazada; DeepSeek local sí supera descriptivamente a Llama y a heurísticos de frontera.
2. **Evidencia para Triton Heal:** Dual reduce FNR vs. Llama (C3, Holm); trade-off en F1 por falsos positivos.
3. **Latencia:** Local Ollama (2–8 s) vs. heurístico de frontera (~ 80 ms); no comparable con WAN real.
4. **Reproducibilidad:** `make experiments stats figures pdf`; entorno M4 documentado en Tabla 3.
5. **Escala:** Comparación 8B vs 16B local realizada; frontera comercial pendiente de API.

A. Configuración planeada vs. corrida real

Cuadro 6: Backends por configuración (mayo 2026).

Configuración	Planeado	Ejecutado
solo_llama	Ollama Llama-3.1-8B	Ollama llama3.1:8b (Metal / M4)
solo_deepseek	Ollama DeepSeek-Coder-V2	Ollama deepseek-coder-v2 (8.9 GB, M4)
solo_claude	API Claude 3.5 Sonnet	Heurístico (sin API key)
solo_gpt4o	API GPT-4o	Heurístico (sin API key)
triton_heal_dual	Llama + veto Claude	Ollama Llama + verifier_strict (veto local)

B. Reproducibilidad

```
cd Estadística/triton_heal_experiment
.venv/bin/python -m src.stats_analysis
.venv/bin/python -m src.plot_results
```

```
cd ../report && tectonic main.tex
```

Archivos: results/predictions.csv, results/stats_summary.json, report/tables/*.tex, figures/*.pdf.

C. Referencias

- Tillet, P., et al. (2019). Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations.
- McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions.
- Holm, S. (1979). A simple sequentially rejective multiple test procedure.